



Klasyfikacja meczów League of Legends przy ograniczonych danych z pierwszych 10 minut meczu

Michał Wojda* (227753), Sebastian Śmietański* (227914),
Tomasz Teofilak* (227674), Tomasz Żołądek* (230998)

*Wydział Zastosowań Informatyki i Matematyki, Szkoła Główna Gospodarstwa Wiejskiego, Warszawa, Polska

(s227753@sggw.edu.pl, s227914@sggw.edu.pl, s227674@sggw.edu.pl, s230998@sggw.edu.pl)

Otrzymano: 09.06.2026 Zaakceptowano: 09.06.2027

Streszczenie- Lorem ipsum dolor sit amet, consectetur adipiscing elit. Cras ex odio, elementum sit amet quam ac, accumsan pharetra libero. Suspendisse odio nisi, consequat sed ornare sed, convallis sed nulla. Nunc nec enim ut nisi facilisis cursus. Cras nec enim neque. Quisque nec consectetur odio. Pellentesque eget euismod sapien. Cras urna urna, gravida a eros eget, tincidunt scelerisque metus. Ut eget mi vehicula augue bibendum dapibus. Ut tempus tellus eros, eu lobortis est dignissim sit amet. Integer id molestie dolor. Mauris tincidunt nunc erat, ut suscipit lacus pharetra varius. Sed quis neque ut ligula vestibulum egestas a sed felis. Quisque vulputate tincidunt metus.

Słowa kluczowe: League of Legends, LoL, klasyfikacja, predykcja wyniku meczu, NAZWY UŻYTYCH METOD

Spis treści

1 Wprowadzenie	3
1.1 Wyjaśnienie terminów	3
2 Przedmiot badania	4
2.1 Cel i zakres badania	4
2.2 Źródło danych	4
2.3 Przegląd literatury	5
3 Wstępna analiza danych	7
3.1 Statystyki opisowe	7
3.2 Braki danych	8
3.3 Wartości odstające	8
3.4 Standaryzowanie danych	8
3.5 Usuwanie cech	9
3.5.1 Quasi Stałe	9
3.5.2 Skorelowane prosto, liniowo	9
3.5.3 Skorelowane wielokrotnie, liniowo	12



3.6	Łączenie zmiennych	13
3.6.1	Agregacja	14
3.6.2	PCA	14
4	Analiza końcowego zbioru danych	17
4.1	Wyjaśnienie zmiennych	17
4.2	Charakterystyka zmiennych	18
4.3	Statystyki opisowe	19
4.4	Wizualizacja	19
5	Opisy metod	24
5.1	Sieci neuronowe	24
5.1.1	Opis	24
5.1.2	Procedura uczenia i ewaluacji	25
5.2	Drzewo klasyfikacyjne	25
5.2.1	Opis	25
5.2.2	Kryterium podziału	25
5.2.3	Przycinanie drzewa	26
5.3	Sieci bayesowskie	26
5.3.1	Opis	26
5.3.2	Uczenie struktury i parametrów	26
5.3.3	Predykcja modelu	27
5.4	Opis metody hybrydowej	27
5.5	Metody oceniania	27
5.5.1	Macierz pomyłek i miary klasyfikacji	27
5.5.2	Krzywa ROC oraz pole pod krzywą AUC	28
6	Rezultaty	29
6.1	Rezultat metody 1	29
6.2	Rezultat metody 2	29
6.3	Rezultat metody 3	29
6.4	Rezultat metody hybrydowej	29
6.5	Porównanie skuteczności metod	29
6.6	Użycie najlepszego modelu na syntetycznych danych	29
7	Podsumowanie	29
8	Załączniki	29



1. Wprowadzenie

[TODO][USUNAC]musi być przynajmniej jedna cytacja żeby latex się nie wywalił [1]

League of Legends jest jedną z najpopularniejszych gier komputerowych typu MOBA (Multiplayer Online Battle Arena), w której dwie pięcioosobowe drużyny rywalizują ze sobą w celu zniszczenia głównej struktury przeciwnika, zwanej Nexusem. Rozgrywka opiera się na współpracy zespołowej, zdobywaniu zasobów oraz stopniowym budowaniu przewagi ekonomicznej i strategicznej nad drużyną przeciwną.

Na przebieg meczu wpływa wiele wzajemnie powiązanych czynników, takich jak liczba eliminacji, zdobywane złoto, doświadczenie bohaterów, kontrola mapy czy przejmowanie neutralnych obiektów. Szczególnie istotna jest początkowa faza rozgrywki, ponieważ to właśnie w pierwszych minutach meczu budowane są podstawy dalszej przewagi, które często mają wpływ na końcowy rezultat spotkania.

Współczesne gry sieciowe generują duże ilości danych opisujących przebieg rozgrywek, co umożliwia wykorzystanie metod analizy danych oraz uczenia maszynowego do badania zależności występujących podczas meczu. Analiza statystyk z wczesnego etapu gry może dostarczyć informacji o typowych schematach prowadzących do zwycięstwa lub porażki drużyny.

Problem badawczy poruszany w niniejszej pracy dotyczy możliwości przewidywania wyniku meczu rankingowego na podstawie statystyk z pierwszych 10 minut rozgrywki. W pracy wykorzystano dane dotyczące meczów rankingowych w celu zbadania, czy metody klasyfikacji pozwalają przewidywać zwycięstwo lub porażkę drużyny niebieskiej na podstawie cech opisujących początkową fazę meczu. Dodatkowym celem jest określenie, które zmienne mają największy wpływ na skuteczność modeli klasyfikacyjnych.

1.1. Wyjaśnienie terminów

- **Doświadczenie** - Doświadczenie (experience, XP) jest zasobem zdobywanym przez bohaterów podczas rozgrywki głównie poprzez przebywanie w pobliżu eliminowanych przeciwników, takich jak miniony czy potwory z dżungli. Zdobycie doświadczenia pozwala bohaterom awansować na kolejne poziomy, co zwiększa ich statystyki oraz umożliwia rozwijanie umiejętności.
- **Złoto** - Złoto jest podstawową walutą występującą w grze. Może być zdobywane między innymi poprzez eliminowanie minionów, bohaterów przeciwnika, potworów z dżungli oraz niszczenie struktur. Zgromadzone złoto pozwala na zakup przedmiotów zwiększających siłę bohatera.
- **Śmierci, zabójstwa i asysty** - Zabójstwo oznacza wyeliminowanie bohatera drużyny przeciwniej. Drużyna zdobywa wówczas złoto oraz przewagę czasową wynikającą z chwilowej nieobecności przeciwnika na mapie. Śmierć oznacza utratę bohatera na określony czas, co ogranicza możliwości drużyny. Asysta przyznawana jest graczom, którzy uczestniczyli w eliminacji przeciwnika, lecz nie zadali decydującego ciosu.
- **First Blood** - First Blood oznacza pierwsze zabójstwo wykonane w meczu. Zdarzenie to zapewnia dodatkową nagrodę w postaci złota i często umożliwia zdobycie wczesnej przewagi nad przeciwnikiem.
- **Wardy** - Wardy są przedmiotami lub umiejętnościami zapewniającymi wizję wybranego obszaru mapy. Mogą być rozmieszczane przez graczy w celu kontrolowania ruchów przeciwnika oraz zabezpieczania ważnych obiektów. Wrogie wardy mogą zostać wykryte i zniszczone przez drużynę przeciwną, co ogranicza dostęp przeciwników do informacji o mapie.
- **Miniony** - Miniony są jednostkami sterowanymi przez komputer, które regularnie pojawiają się w bazach obu drużyn i poruszają się wyznaczonymi liniami mapy. Eliminowanie minionów stanowi jedno z głównych źródeł złota i doświadczenia podczas wczesnej fazy gry.
- **Miniony z dżungli** - Miniony z dżungli, nazywane również potworami neutralnymi, występują w obszarze dżungli pomiędzy liniami. Ich eliminowanie zapewnia złoto, doświadczenie oraz dodatkowe efekty wzmacniające bohaterów.
- **Smoki** - Smoki są neutralnymi potworami pojawiającymi się okresowo na mapie. Ich pokonanie zapewnia drużynie trwałą premie wzmacniającą statystyki bohaterów. Kontrola smoków jest jednym z ważniejszych elementów strategicznych rozgrywki.



- **Heroldy** - Heroldy, określane również jako Rift Herald, są neutralnymi potworami pojawiającymi się we wczesnej i środkowej fazie gry. Po pokonaniu herolda drużyna może przyzwać go na wybranej linii, gdzie pomaga on w niszczeniu struktur przeciwnika.

2. Przedmiot badania

Przedmiotem badania są mecze rankingowe gry League of Legends analizowane na podstawie statystyk pochodzących z pierwszych 10 minut rozgrywki. W pracy skupiono się na zależności pomiędzy wczesną przewagą jednej z drużyn a końcowym wynikiem meczu. Analiza obejmuje zastosowanie metod klasyfikacji w celu sprawdzenia, czy dane dostępne na początkowym etapie gry pozwalają skutecznie przewidywać zwycięstwo lub porażkę drużyny niebieskiej.

2.1. Cel i zakres badania

Celem badania jest klasyfikacja meczów League of Legends na podstawie danych z pierwszych 10 minut rozgrywki przy użyciu kilku metod klasyfikacji, a następnie ocena skuteczności tych metod w przewidywaniu rzeczywistego wyniku meczu, rozumianego jako wygrana lub przegrana drużyny niebieskiej.

Istotnym elementem pracy jest porównanie wyników uzyskanych za pomocą trzech metod klasyfikacji: [TODO][NAZWA METODY 1], [NAZWA METODY 2] oraz [NAZWA METODY 3], a także metody mieszanej [NAZWA METODY MIESZANEJ], łączącej cechy wybranych podejść. Celem analizy jest wskazanie, które z zastosowanych rozwiązań najlepiej przewiduje wynik meczu na podstawie danych dostępnych na wczesnym etapie gry.

Analiza ma charakter predykcyjny i koncentruje się na sprawdzeniu, czy informacje dostępne w początkowej fazie meczu pozwalają na skuteczne przewidywanie jego końcowego rezultatu.

Zakres badania obejmuje:

- charakterystykę oraz analizę zbioru danych zawierającego statystyki meczów League of Legends,
- przeprowadzenie wstępnej analizy danych, obejmującej statystyki opisowe oraz analizę korelacji pomiędzy zmiennymi,
- przygotowanie danych do procesu uczenia modeli, w tym obsługę wartości odstających, analizę braków danych, skalowanie cech oraz usunięcie zbędnych zmiennych,
- analizę końcowego zbioru danych po przeprowadzeniu preprocessingu,
- przedstawienie teoretycznych podstaw zastosowanych metod klasyfikacji,
- implementację oraz zastosowanie metod klasyfikacji: [TODO][NAZWA METODY 1], [NAZWA METODY 2], [NAZWA METODY 3] oraz [NAZWA METODY HYBRYDOWEJ],
- ocenę skuteczności modeli przy użyciu wybranych miar jakości klasyfikacji,
- analizę i interpretację uzyskanych wyników,
- porównanie skuteczności zastosowanych metod klasyfikacji,
- wykorzystanie najlepszego modelu do klasyfikacji syntetycznie wygenerowanych danych testowych.

Analizę danych oraz implementację modeli klasyfikacyjnych wykonano przy użyciu języka R. Obliczenia przeprowadzono w środowisku RStudio. Użyte biblioteki: [TODO][LISTA WSZYSTKICH BIBLIOTEK]

2.2. Źródło danych

W pracy wykorzystano publicznie dostępny zbiór danych zawierający statystyki meczów rankingowych gry League of Legends. Dane pochodzą z serwisu Kaggle i zostały pozyskane przy użyciu oficjalnego API udostępnianego przez Riot Games.



Zbiór obejmuje mecze rankingowe rozgrywane na wysokim poziomie rankingowym tj. od rangi Diamond wzwyż, co odpowiada około 5% najlepszych graczy. Dane pochodzą z okresu sprzed około sześciu lat. Autor zbioru nie określa dokładnego regionu serwerowego, z którego zostały pobrane mecze.

Pojedynczy rekord reprezentuje statystyki obu drużyn po upływie pierwszych 10 minut pojedynczego meczu. Dane opisują sytuację ekonomiczną drużyn, przebieg walk, kontrolę mapy oraz realizację celów neutralnych we wczesnej fazie rozgrywki.

Zbiór danych składa się z 9879 rekordów oraz 40 zmiennych. Zmienną decyzyjną jest blueWins, określająca końcowy wynik meczu z perspektywy drużyny niebieskiej. Wartość zmiennej informuje, czy drużyna niebieska odniosła zwycięstwo.

Klasy w zbiorze danych są w przybliżeniu zbalansowane, co oznacza, że liczba zwycięstw drużyny niebieskiej i czerwonej jest podobna.

2.3. Przegląd literatury

[TODO][DODAC CYTOWNIA DO PRAC Z KAGGLA] Ze względu na ograniczoną liczbę prac dotyczących wczesnej fazy gry w League of Legends, przegląd oparto głównie na dwóch analizach z Kaggle wykorzystujących ten sam zbiór danych.

Pierwsza praca [?] pokazuje, że już w pierwszych 10 minutach meczu pojawiają się istotne różnice statystyczne między drużynami, które mogą wpływać na dalszy przebieg rozgrywki. Wyniki sugerują, że wczesna faza gry dostarcza informacji o potencjalnym wyniku meczu.

Druga analiza [?] wskazuje, że takie zmienne jak liczba zabójstw, asyst, złoto, doświadczenie i zabite stwory dobrze różnicują mecze wygrane i przegrane. Potwierdza to, że przewaga budowana na początku gry często przekłada się na końcowy rezultat.

Trzecia praca [?] zawiera dokładną analizę wszystkich zmiennych oraz ich wpływu na wynik meczu.

Wnioski przedstawione w analizowanych pracach uzasadniają zastosowane w niniejszej pracy podejście badawcze. Dotychczasowe analizy wskazują, że dane z początkowej fazy rozgrywki zawierają istotną informację predykcyjną, a wykorzystanie metod klasyfikacji może umożliwić skuteczne przewidywanie wyniku meczu na podstawie ograniczonego zestawu cech opisujących pierwsze minuty gry.





3. Wstępna analiza danych

3.1. Statystyki opisowe

Zmienna	Średnia	Mediana	Min.	Maks.	Odch. std.	Skośność
blueWardsPlaced	22.29	16.0	5.0	250.0	18.02	4.14
blueWardsDestroyed	2.82	3.0	0.0	27.0	2.17	2.85
blueFirstBlood	0.50	1.0	0.0	1.0	0.50	-0.02
blueKills	6.18	6.0	0.0	22.0	3.01	0.54
blueDeaths	6.14	6.0	0.0	22.0	2.93	0.51
blueAssists	6.65	6.0	0.0	29.0	4.06	0.89
blueEliteMonsters	0.55	0.0	0.0	2.0	0.63	0.69
blueDragons	0.36	0.0	0.0	1.0	0.48	0.57
blueHeralds	0.19	0.0	0.0	1.0	0.39	1.60
blueTowersDestroyed	0.05	0.0	0.0	4.0	0.24	5.59
blueTotalGold	16503.46	16398.0	10730.0	23701.0	1535.45	0.47
blueAvgLevel	6.92	7.0	4.6	8.0	0.31	-0.34
blueTotalExperience	17928.11	17951.0	10098.0	22224.0	1200.52	-0.25
blueTotalMinionsKilled	216.70	218.0	90.0	283.0	21.86	-0.27
blueTotalJungleMinionsKilled	50.51	50.0	0.0	92.0	9.90	0.12
blueGoldDiff	14.41	14.0	-10830.0	11467.0	2453.35	0.03
blueExperienceDiff	-33.62	-28.0	-9333.0	8348.0	1920.37	0.02
blueCSPerMin	21.67	21.8	9.0	28.3	2.19	-0.27
blueGoldPerMin	1650.35	1639.8	1073.0	2370.1	153.54	0.47
redWardsPlaced	22.37	16.0	6.0	276.0	18.46	4.56
redWardsDestroyed	2.72	2.0	0.0	24.0	2.14	2.95
redFirstBlood	0.50	0.0	0.0	1.0	0.50	0.02
redKills	6.14	6.0	0.0	22.0	2.93	0.51
redDeaths	6.18	6.0	0.0	22.0	3.01	0.54
redAssists	6.66	6.0	0.0	28.0	4.06	0.82
redEliteMonsters	0.57	0.0	0.0	2.0	0.63	0.62
redDragons	0.41	0.0	0.0	1.0	0.49	0.35
redHeralds	0.16	0.0	0.0	1.0	0.37	1.85
redTowersDestroyed	0.04	0.0	0.0	2.0	0.22	5.34
redTotalGold	16489.04	16378.0	11212.0	22732.0	1490.89	0.41
redAvgLevel	6.93	7.0	4.8	8.2	0.31	-0.40
redTotalExperience	17961.73	17974.0	10465.0	22269.0	1198.58	-0.28
redTotalMinionsKilled	217.35	218.0	107.0	289.0	21.91	-0.29
redTotalJungleMinionsKilled	51.31	51.0	4.0	92.0	10.03	0.23
redGoldDiff	-14.41	-14.0	-11467.0	10830.0	2453.35	-0.03
redExperienceDiff	33.62	28.0	-8348.0	9333.0	1920.37	-0.02
redCSPerMin	21.73	21.8	10.7	28.9	2.19	-0.29
redGoldPerMin	1648.90	1637.8	1121.2	2273.2	149.09	0.41

Table 1. Statystyki opisowe wszystkich zmiennych.



3.2. Braki danych

W analizowanym zbiorze danych nie stwierdzono braków danych.

3.3. Wartości odstające

W celu ograniczenia wpływu wartości odstających na działanie modeli klasyfikacyjnych przeprowadzono proces oczyszczania danych. Za wartości odstające uznano jedynie 1% najwyższych wartości zmiennych:

- blueWardsPlaced
- redWardsPlaced

jako że te zmienne miały największą ilość skrajnie dużych wartości. Dodatkowo, takie wartości świadczą o braku poważnego traktowania gry ze strony drużyny charakteryzującej się wysoką wartością zmiennej *WardsPlaced*.

Dlatego dla każdej wyżej wymienionej cechy wyznaczano górny próg odpowiadający wybranemu percentylowi:

$$Q_g = Q_{1-\frac{p}{100}}$$

gdzie:

- Q_g — górny próg dla danej cechy,
- Q — funkcja percentylowa,
- p — procent najwyższych wartości uznawanych za odstające.

Następnie wszystkie wartości większe od wyznaczonego progu były oznaczane jako brakujące:

$$x = \begin{cases} x, & \text{gdy } x \leq Q_g \\ NA, & \text{gdy } x > Q_g \end{cases}$$

Po oznaczeniu wartości odstających jako brakujące usuwano wszystkie rekordy zawierające przynajmniej jedną wartość NA. Ostatecznie ze zbioru danych usunięto 190 rekordów, co stanowiło w przybliżeniu 1.92% wszystkich obserwacji.

Zastosowana metoda pozwoliła ograniczyć wpływ skrajnych obserwacji na proces uczenia modeli oraz zmniejszyć ryzyko zaburzenia działania algorytmów przez nietypowe wartości.

3.4. Standaryzowanie danych

W celu ujednolicenia rozkładów oraz zakresów wartości poszczególnych cech zastosowano standaryzację danych z wykorzystaniem funkcji `scale()` dostępnej w języku R. Metoda ta polega na przekształceniu wartości zmiennych w taki sposób, aby każda cecha posiadała średnią równą 0 oraz odchylenie standardowe równe 1.

Standaryzacja została wykonana zgodnie ze wzorem:

$$x' = \frac{x - \mu}{\sigma}$$

gdzie:

- x — wartość przed skalowaniem,
- μ — średnia wartość danej cechy,



- σ — odchylenie standardowe danej cechy,
- x' — wartość po standaryzacji.

W wyniku zastosowania standaryzacji wszystkie analizowane cechy numeryczne zostały sprowadzone do wspólnej skali. Pozwoliło to ograniczyć wpływ różnic pomiędzy zakresami wartości poszczególnych zmiennych na działanie modeli klasyfikacyjnych oraz poprawić stabilność procesu uczenia.

3.5. Usuwanie cech

Ta podsekcja będzie poświęcona procesowi doboru cech poprzez usuwanie Quasi stałych oraz usuwanie lub łączenie względnie silnie skorelowanych.

3.5.1. Quasi Stałe

Do detekcji Quasi stałych zmiennych wykorzystaliśmy funkcję z pakietu *Caret* o nazwie *nearZeroVar*. Sprawdza ona, czy istnieją jakieś zmienne, które przyjmują tylko jedną unikalną wartość lub mają bardzo mało unikalnych wartości relatywnie i stosunek częstotliwości najczęściej występującej wartości do częstotliwości drugiej najczęściej występującej wartości jest duży, na co wskazuje sama nazwa funkcji. Po jej wykonaniu otrzymaliśmy następującą listę cech:

- blueTowersDestroyed
- redTowersDestroyed

Otrzymaliśmy więc cechę "TowersDestroyed" dla obu drużyn. Dodatkowo z tabeli 1 wiemy, że ich mediany wynoszą 0. Nie powinno to zaskakiwać, jako że w pierwszych 10 minutach rozgrywki - względnie początkowej fazie - rzadko się zdarza zniszczenie wieży przeciwnika. W związku z powyższym usuwamy te 2 cechy z naszego data setu

3.5.2. Skorelowane prosto, liniowo

Cechy skorelowane prosto, liniowo, to takie, pomiędzy którymi istnieje korelacja cecha do cechy. Ergo w tej podsekcji zajmiemy się znajdowaniem takich par i usuwaniu jednej z cech na tej podstawie. Wykorzystamy do tego wbudowaną funkcję w R o nazwie "cor". Za jej pomocą stworzymy macierz korelacji. Wizualizacji dokonamy z wykorzystaniem funkcji "corrplot" z pakietu o tej samej nazwie.

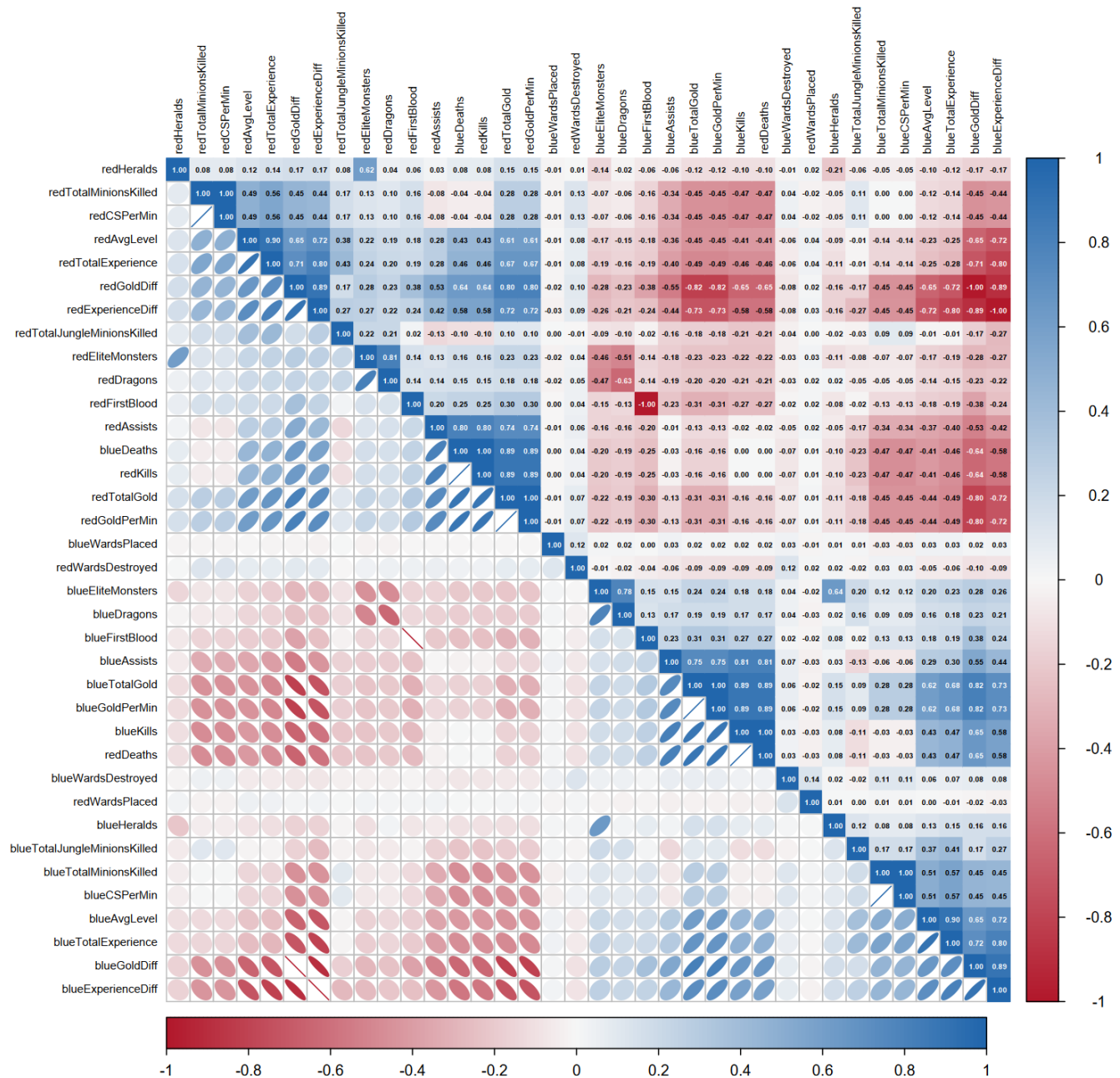


Fig. 1. Macierz korelacji dla wszystkich zmiennych bez quasi stałych 1.



Powyższa macierz przedstawia korelacje liniowe, proste pomiędzy wszystkimi zmiennymi, jakie nam zostały po usunięciu quasi stałych. Na jego podstawie możemy znaleźć bardzo silnie skorelowane, czyli takie, których poziom korelacji jest większy bądź równy 0.9, pary cech:

Cecha A	Cecha B	Korelacja A i B
blueFirstBlood	redFirstBlood	-1
redDeaths	blueKills	1
blueDeaths	redKills	1
blueTotalGold	blueGoldPerMin	1
blueAvgLevel	blueTotalExperience	0.901297
blueTotalMinionsKilled	blueCSPerMin	1
blueGoldDiff	redGoldDiff	-1
blueExperienceDiff	redExperienceDiff	-1
redTotalGold	redGoldPerMin	1
redAvgLevel	redTotalExperience	0.901748
redTotalMinionsKilled	redCSPerMin	1

Table 2. Tabela korelacji silnie skorelowanych zmiennych 1.



Na podstawie powyższego możemy usunąć następujące cechy;

- blueFirstBlood
- redDeaths
- blueDeaths
- blueTotalGold
- blueAvgLevel
- blueTotalMinionsKilled
- blueGoldDiff
- blueExperienceDiff
- redTotalGold
- redAvgLevel
- redTotalMinionsKilled

3.5.3. Skorelowane wielokrotnie, liniowo

Dodatkowo, możemy, korzystając z definicji poszczególnych cech, zebrać korelacje wielokrotne, liniowe; innymi słowy takie korelacje, w których to jedna zmienna jest skorelowana liniowo z więcej niż jedną. Takich relacji mamy aż 6 i są one przedstawione poniżej:

- $blueEliteMonsters = blueDragons + blueHeralds$
- $redEliteMonsters = redDragons + redHeralds$
- $blueGoldDiff = blueTotalGold - redTotalGold$ gdzie blueGoldDiff został już usunięty
- $redGoldDiff = redTotalGold - blueTotalGold$
- $blueExperienceDiff = blueTotalExperience - redTotalExperience$ gdzie blueExperienceDiff został już usunięty
- $redExperienceDiff = redTotalExperience - blueTotalExperience$

Na podstawie wyżej wymienionych relacji usuwamy następujące zmienne:

- $blueEliteMonsters$ i $redEliteMonsters$ jako że istnieje potencjalnie istotna różnica pomiędzy zabiciem heralda a smoka i dlatego wolimy zostawić zmienne z wymienionymi skorelowane
- $blueGoldPerMin$ i $redGoldPerMin$ jako że są one liniowo skorelowane odpowiednio z $blueTotalGold$ i $redTotalGold$ oraz przez wzgląd na to, że uznajemy różnicę między nimi za istotniejszą od tych zmiennych samych w sobie
- $blueTotalExperience$ i $redTotalExperience$ jako że uznajemy różnicę między nimi za istotniejszą od tych zmiennych samych w sobie



3.6. Łączenie zmiennych

Po poprzedniej podsekcji zostały nam cechy przedstawione na narysowanej poniżej macierzy korelacji:

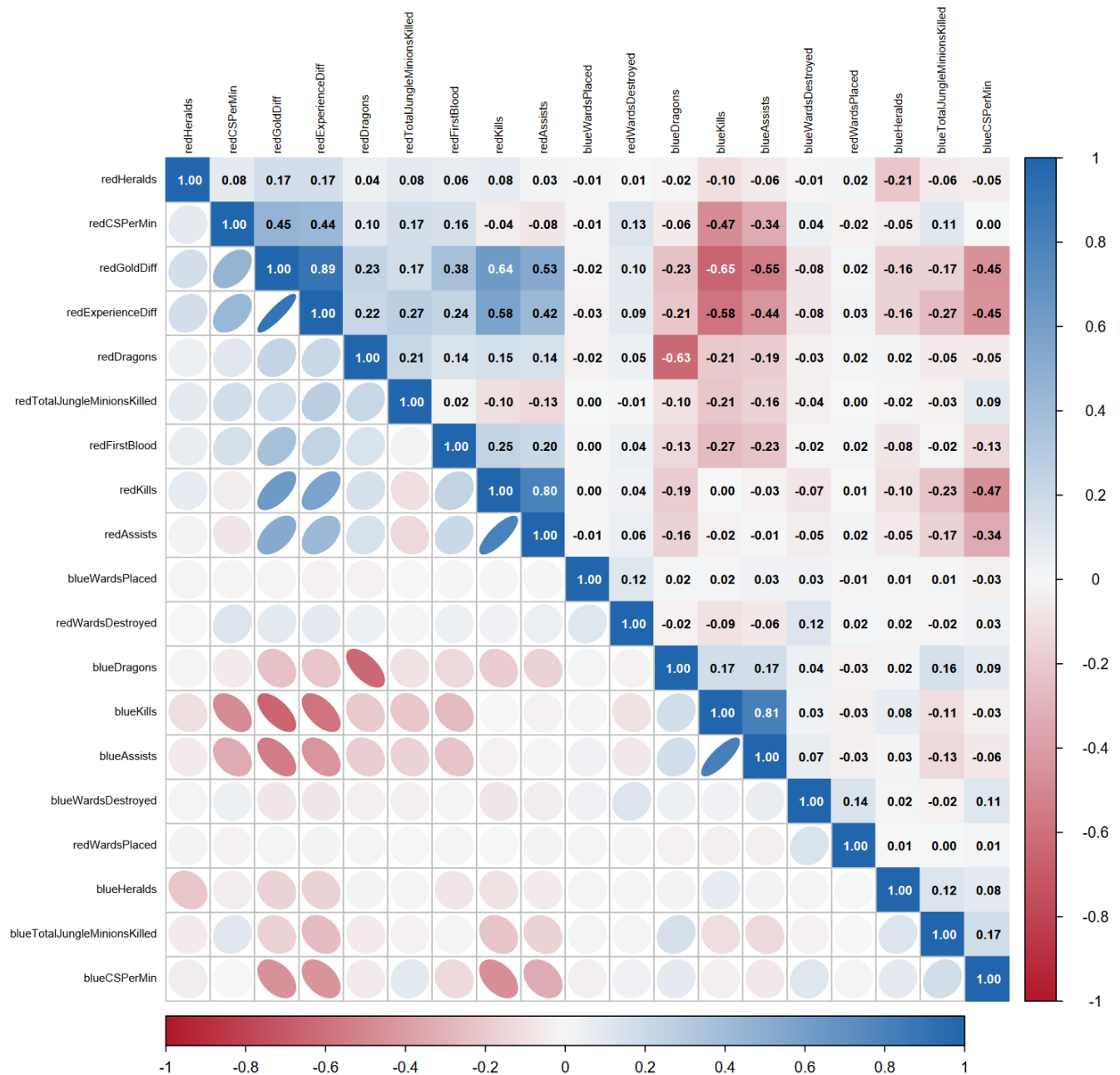


Fig. 2. Macierz korelacji dla wszystkich zmiennych po usuwaniu liniowo skorelowanych



3.6.1. Agregacja

Jak można wyczytać z powyższej macierzy, mamy 3 pary silnie skorelowanych cech. Spośród nich 2 to następujące pary:

Cecha A	Cecha B	Korelacja A i B
redKills	redAssists	0.8
blueKills	blueAssists	0.81

Table 3. Tabela korelacji silnie skorelowanych zmiennych 2.

Te cechy zmagujemy liniowo w jedną, dla każdej drużyny, wedle następującej reguły:

$$KA = Kills + Assists$$

Tym samym otrzymujemy następujące nowe cechy:

- blueKA
- redKA

3.6.2. PCA

Po agregacji została nam następująca para cech silnie skorelowanych:

$$\text{redGoldDiff} \quad \text{redExperienceDiff} \quad 0.89$$

Do zagregowania tych zmiennych wykorzystamy metodę PCA. Poniżej przedstawimy, opis tej metody:

Niech macierz danych będzie opisana jako

$$\mathbf{X} \in \mathbb{R}^{n \times p},$$

gdzie n oznacza liczbę obserwacji, a p liczbę zmiennych. Metoda PCA przekształca pierwotny zbiór p skorelowanych zmiennych w nowy zbiór zmiennych, zwanych **głównymi składowymi**, które są wzajemnie nieskorelowane i uporządkowane według malejącej wariancji.

W pierwszym kroku dane są zwykle centrowane, a w przypadku zmiennych mierzonych w różnych skalach również standaryzowane. Następnie wyznacza się macierz kowariancji:

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X},$$

lub, w przypadku danych standaryzowanych, macierz korelacji.

Kolejnym etapem jest wyznaczenie wartości własnych i wektorów własnych macierzy $\mathbf{S}$:

$$\mathbf{S} \mathbf{v}_k = \lambda_k \mathbf{v}_k,$$

gdzie λ_k oznacza k -tą wartość własną, a $\mathbf{v}_k$ odpowiadający jej wektor własny. Wektory własne wyznaczają kierunki głównych składowych, natomiast wartości własne informują o ilości wariancji wyjaśnianej przez każdą składową.



Główne składowe wyznacza się jako liniowe kombinacje zmiennych pierwotnych:

$$\mathbf{z}_k = \mathbf{X}\mathbf{v}_k.$$

Pierwsza główna składowa wyjaśnia największą możliwą część wariancji danych, druga – największą część pozostałej wariancji, przy zachowaniu ortogonalności względem pierwszej, itd.

Redukcja wymiarowości polega na wyborze pierwszych m składowych, gdzie $m < p$, tak aby zachować jak największą część całkowitej zmienności danych. Udział wariancji wyjaśnionej przez k -tą składową można obliczyć jako:

$$\frac{\lambda_k}{\sum_{j=1}^p \lambda_j}.$$

Z kolei skumulowany udział wyjaśnionej wariancji dla pierwszych m składowych wynosi:

$$\frac{\sum_{k=1}^m \lambda_k}{\sum_{j=1}^p \lambda_j}.$$

W praktyce wybiera się taką liczbę składowych, aby skumulowana wariancja osiągała założony poziom, np. 80%, 90% lub 95%. Dzięki temu możliwe jest zastąpienie dużej liczby zmiennych mniejszym zbiorem nowych cech, co upraszcza analizę, zmniejsza złożoność modeli oraz ogranicza problem współliniowości.

W skrypcie będziemy bazować na wbudowanej funkcji "prcomp" i do wizualizacji rzutu wykorzystamy funkcję "fviz_pca_biplot" z biblioteki factoextra. Poniżej przedstawiony jest wykres po zastosowaniu PCA.

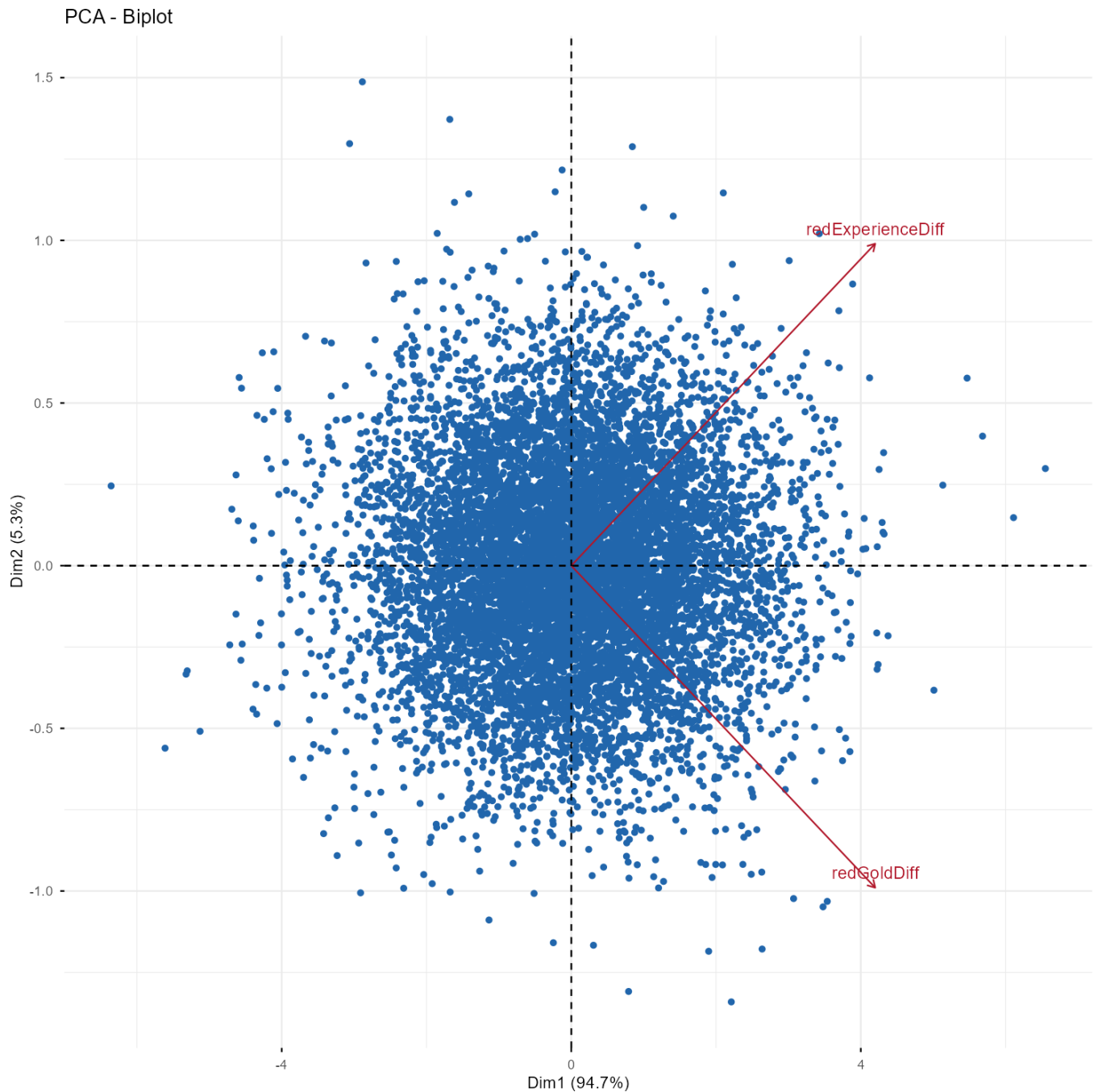


Fig. 3. Wykres PCA dla cech redGoldDiff i redExperienceDiff

Jak na załączonym wyżej obrazku widać, nowa cecha, roboczo nazwana *Dim1*, odpowiada za aż około 94.7% wariancji. Stąd też możemy uznać, że taka agregacja będzie dobrą formą wyeliminowania silnej korelacji. Od tego momentu zmienna robocza nazwana *Dim1* będzie miała nazwę *diff_PCA_Component_1*.

Po wykonaniu wszystkich kroków usuwania silnie i bardzo silnie skorelowanych cech możemy przedstawić jak ostatecznie wygląda macierz korelacji. Dodatkowo macierz uwzględnia cechę *blueWins*, aby dodatkowo ukazać jak silnie jest ona skorelowana ze zmiennymi, które ostatecznie zostały.

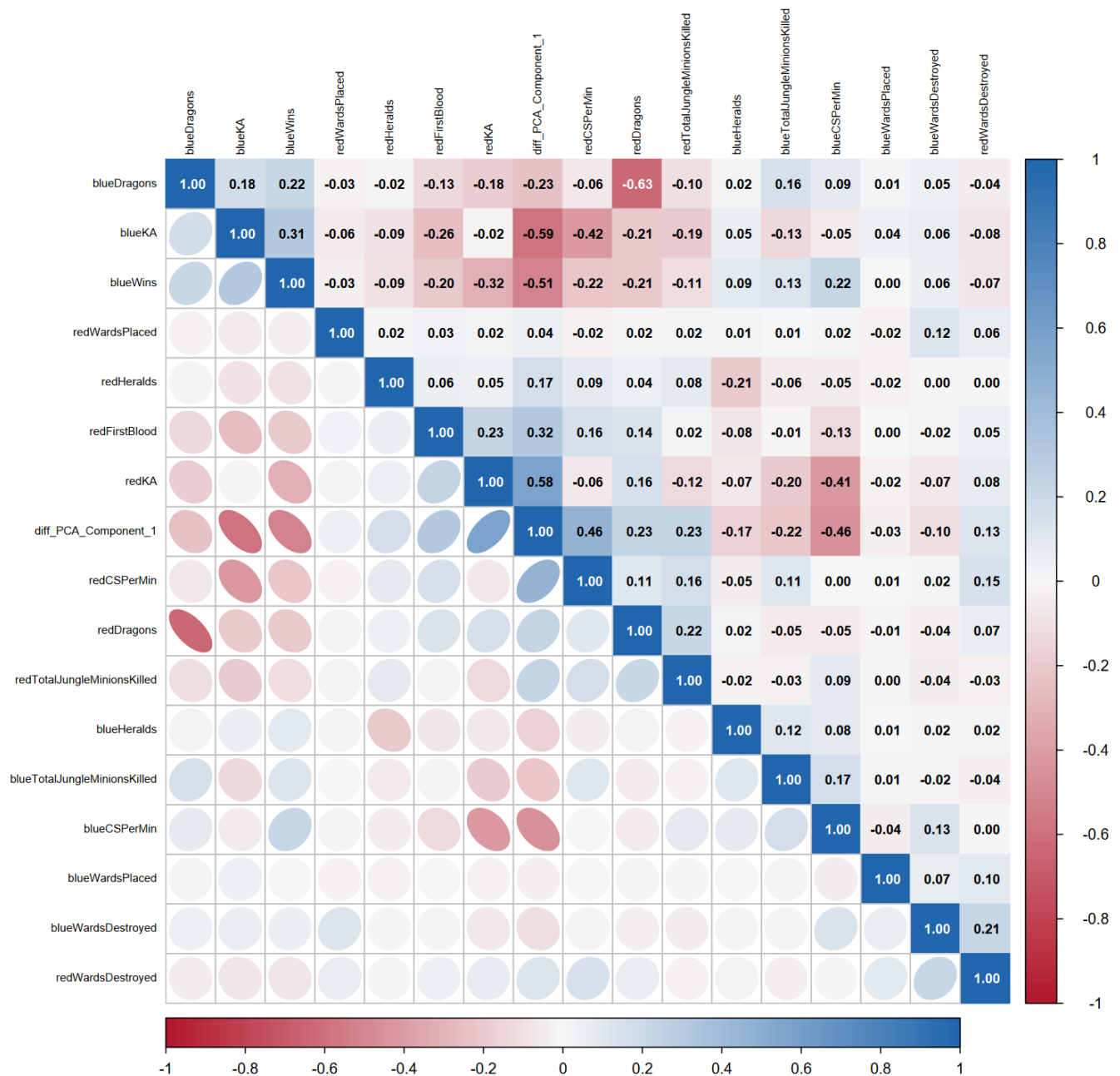


Fig. 4. Macierz korelacji dla wszystkich zmiennych po usuwaniu liniowo skorelowanych

4. Analiza końcowego zbioru danych

4.1. Wyjaśnienie zmiennych

- **blueWardsPlaced** — Ilość wardów postawionych przez drużynę niebieską.
- **blueWardsDestroyed** — Ilość wardów zniszczonych przez drużynę niebieską.



- **blueDragons** — Liczba smoków zabitych przez drużynę niebieską.
- **blueHeralds** — Liczba Heroldów zabitych przez drużynę niebieską.
- **blueTotalJungleMinionsKilled** — Łączna ilość zabitych minionów z dżungli przez drużynę niebieską.
- **blueCSPerMin** — Ilość zabitych minionów przez drużynę niebieską w przeliczeniu na minute.
- **redWardsPlaced** — Ilość wardów postawionych przez drużynę czerwoną.
- **redWardsDestroyed** — Ilość wardów zniszczonych przez drużynę czerwoną.
- **redFirstBlood** — Informacja o zdobyciu pierwszego zabójstwa (First Blood) przez drużynę czerwoną.
- **redDragons** — Liczba smoków zabitych przez drużynę czerwoną.
- **redHeralds** — Liczba Heroldów zabitych przez drużynę czerwoną.
- **redTotalJungleMinionsKilled** — Łączna ilość zabitych minionów z dżungli przez drużynę czerwoną.
- **redCSPerMin** — Ilość zabitych minionów przez drużynę czerwoną w przeliczeniu na minute.
- **blueKA** — Łączna ilość zabójstw i asyst drużyny niebieskiej.
- **redKA** — Łączna ilość zabójstw i asyst drużyny czerwonej.
- **diff_PCA_Component_1** — Nowa zmienna utworzona w procesie PCA ze zmiennych *redGoldDiff* i *redExperienceDiff*.

4.2. Charakterystyka zmiennych

Zmienne opisujące ilość zabitych smoków i heroldów powinny być zmiennymi numerycznymi jednak przez ograniczoną naturę badanego zbioru danych przyjmują jedynie wartości 0 lub 1. Z tego powodu zostały one opisane jako zmienne binarne.

Zmienna	Typ	Skala	Charakter
blueWardsPlaced	Numeryczna	Ilościowa	Skokowa
blueWardsDestroyed	Numeryczna	Ilościowa	Skokowa
blueDragons	Binarna	Nominalna	Skokowa
blueHeralds	Binarna	Nominalna	Skokowa
blueTotalJungleMinionsKilled	Numeryczna	Ilościowa	Skokowa
blueCSPerMin	Numeryczna	Ilościowa	Ciągła
redWardsPlaced	Numeryczna	Ilościowa	Skokowa
redWardsDestroyed	Numeryczna	Ilościowa	Skokowa
redFirstBlood	Binarna	Nominalna	Skokowa
redDragons	Binarna	Nominalna	Skokowa
redHeralds	Binarna	Nominalna	Skokowa
redTotalJungleMinionsKilled	Numeryczna	Ilościowa	Skokowa
redCSPerMin	Numeryczna	Ilościowa	Ciągła
blueKA	Numeryczna	Ilościowa	Skokowa
redKA	Numeryczna	Ilościowa	Skokowa
diff_PCA_Component_1	Numeryczna	Ilościowa	Ciągła

Table 4. Charakterystyka końcowych zmiennych.



4.3. Statystyki opisowe

Zmienna	Średnia	Mediana	Min.	Maks.	Odch. std.	Skośność
blueWardsPlaced	20.97	16.00	5.00	105.00	13.78	3.04
blueWardsDestroyed	2.70	3.00	0.00	11.00	1.73	0.79
blueDragons	0.36	0.00	0.00	1.00	0.48	0.58
blueHeralds	0.19	0.00	0.00	1.00	0.39	1.60
blueTotalJungleMinionsKilled	50.50	50.00	0.00	92.00	9.91	0.11
blueCSPerMin	21.66	21.80	9.00	28.30	2.19	-0.26
redWardsPlaced	20.95	16.00	6.00	104.00	13.43	2.89
redWardsDestroyed	2.60	2.00	0.00	10.00	1.67	0.72
redFirstBlood	0.50	0.00	0.00	1.00	0.50	0.02
redDragons	0.41	0.00	0.00	1.00	0.49	0.35
redHeralds	0.16	0.00	0.00	1.00	0.37	1.86
redTotalJungleMinionsKilled	51.32	51.00	4.00	92.00	10.05	0.23
redCSPerMin	21.73	21.80	10.70	28.90	2.19	-0.29
blueKA	12.84	12.00	0.00	51.00	6.76	0.69
redKA	12.81	12.00	0.00	43.00	6.66	0.63
diff_PCA_Component_1	0.00	0.01	-6.36	6.55	1.38	-0.03

Table 5. Statystyki opisowe końcowych zmiennych.

4.4. Wizualizacja

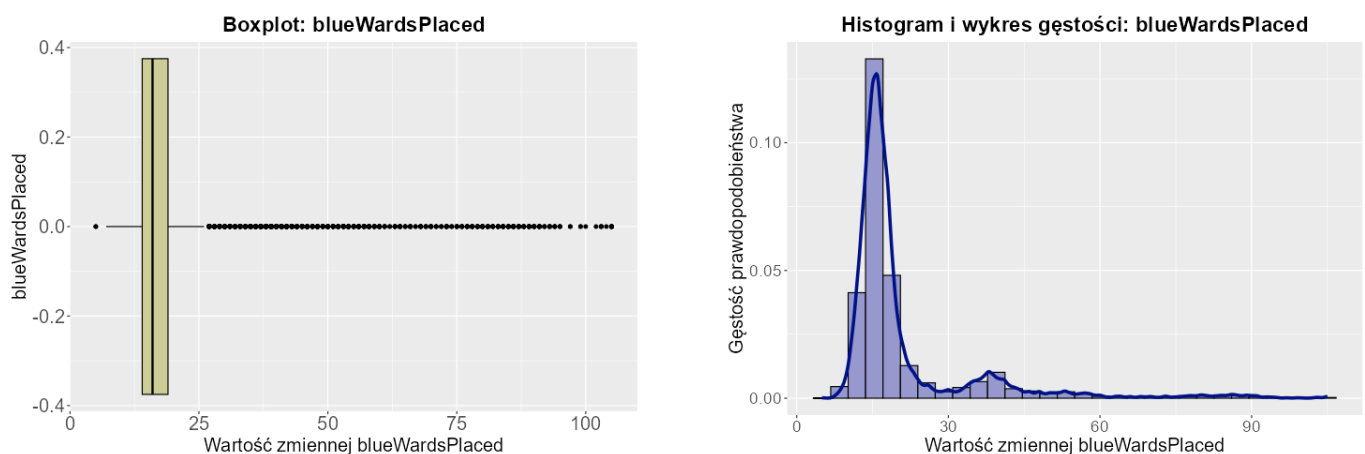


Fig. 5. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej blueWardsPlaced.

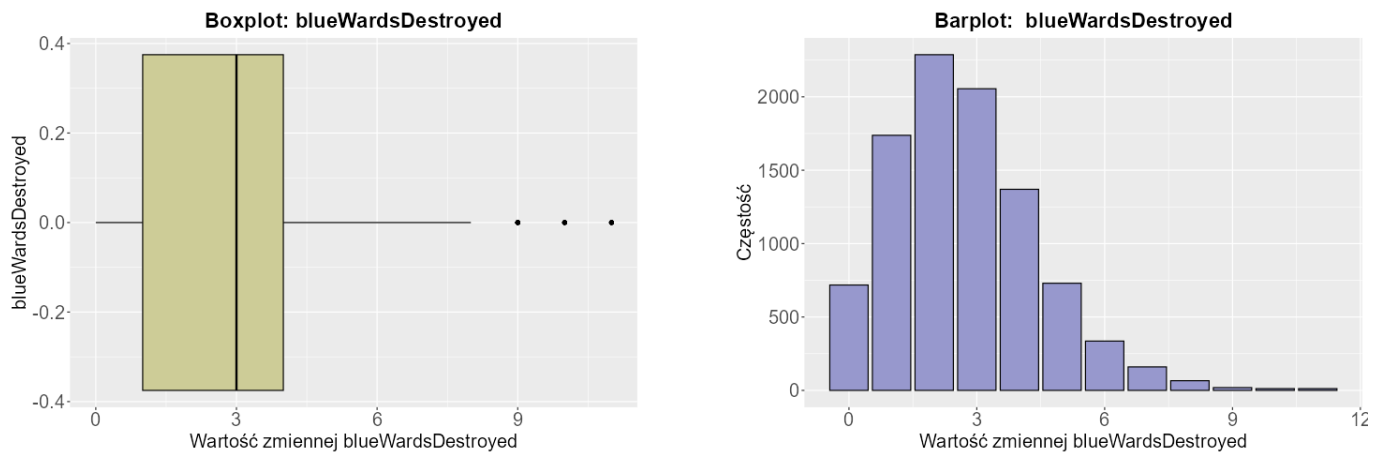


Fig. 6. Boxplot oraz barplot dla zmiennej blueWardsDestroyed.



Fig. 7. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej blueTotalJungleMinionsKilled.

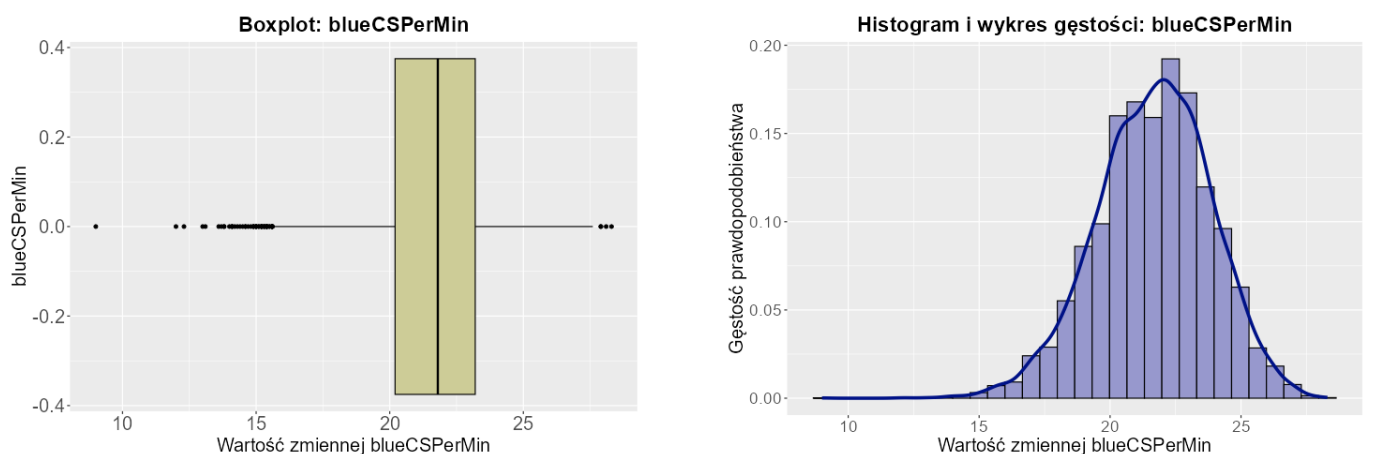


Fig. 8. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej blueCSPerMin.

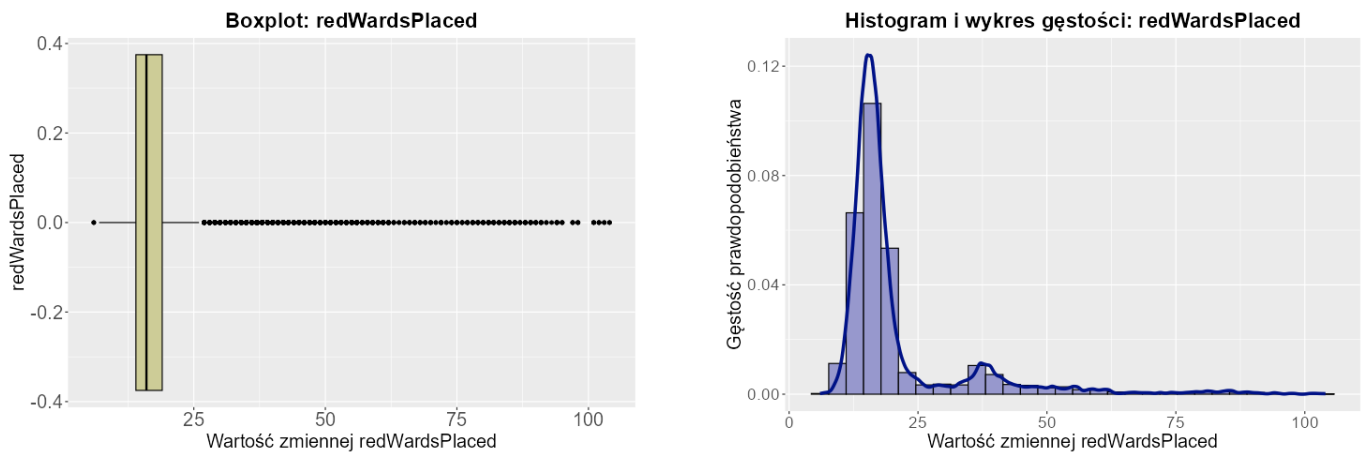


Fig. 9. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej redWardsPlaced.

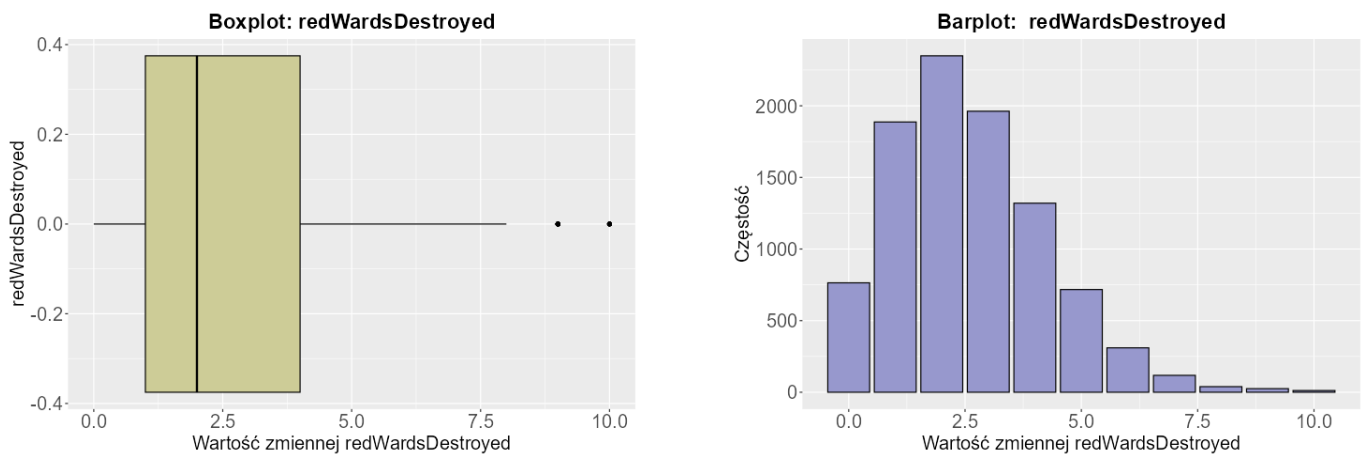


Fig. 10. Boxplot oraz barplot dla zmiennej redWardsDestroyed.



Fig. 11. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej redTotalJungleMinionsKilled.

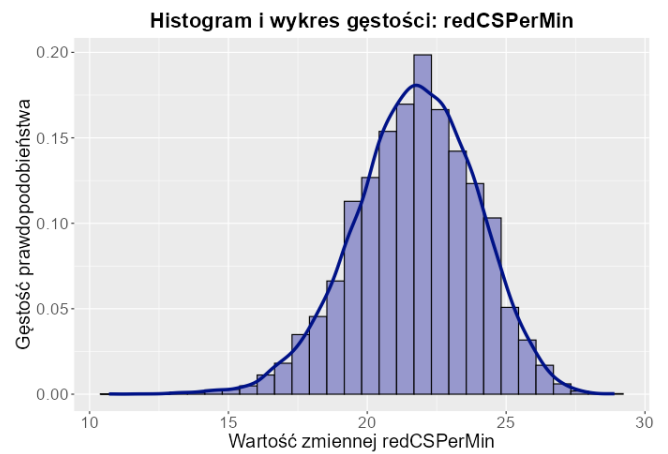
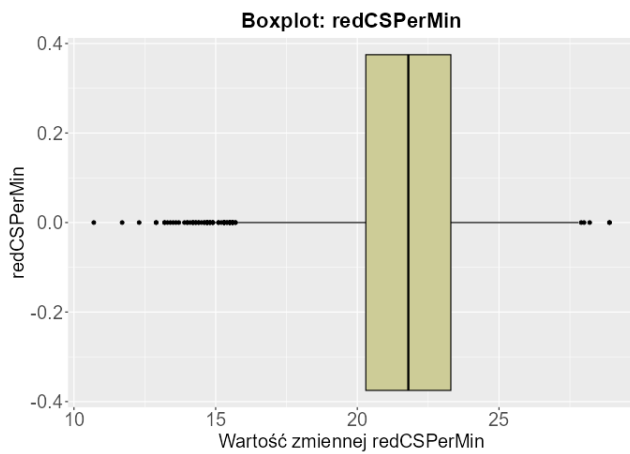


Fig. 12. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej redCSPerMin.

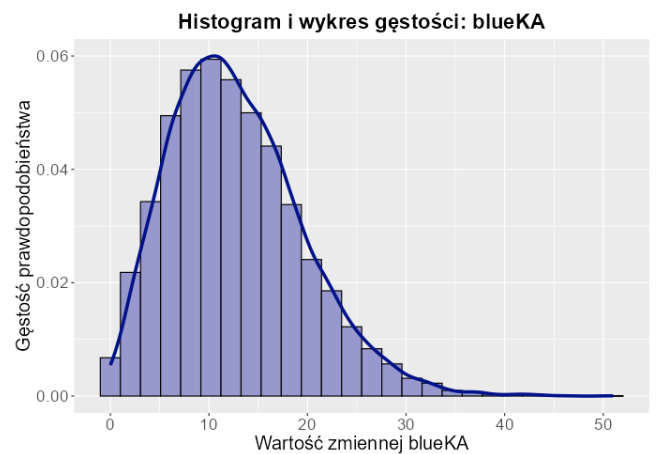
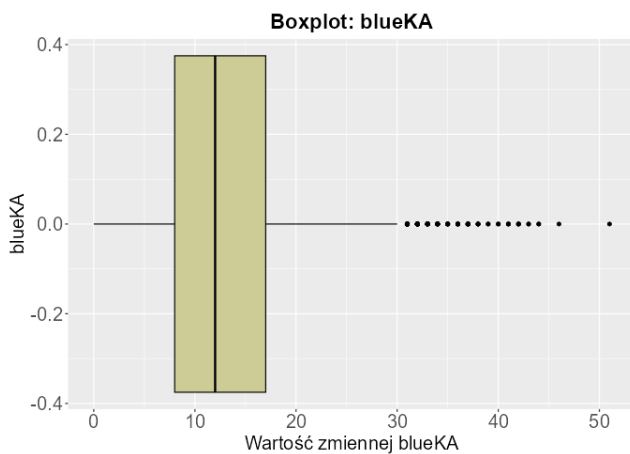


Fig. 13. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej blueKA.

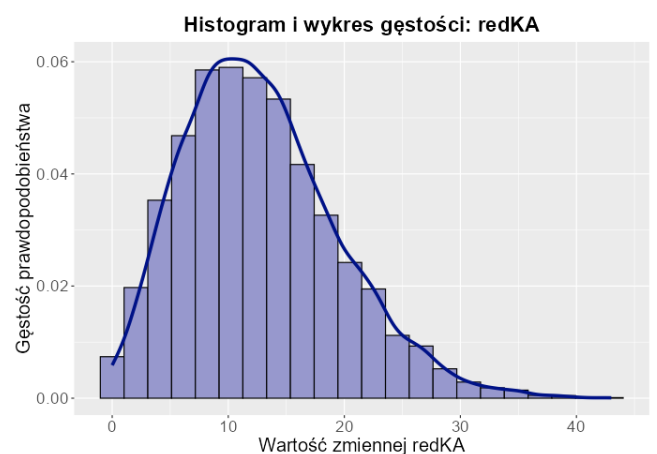
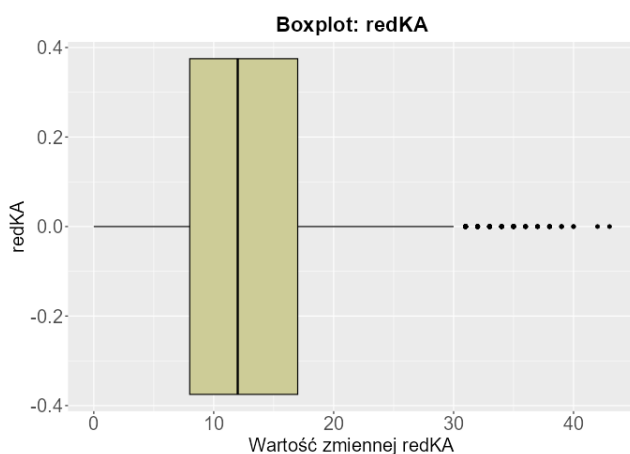


Fig. 14. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej redKA.

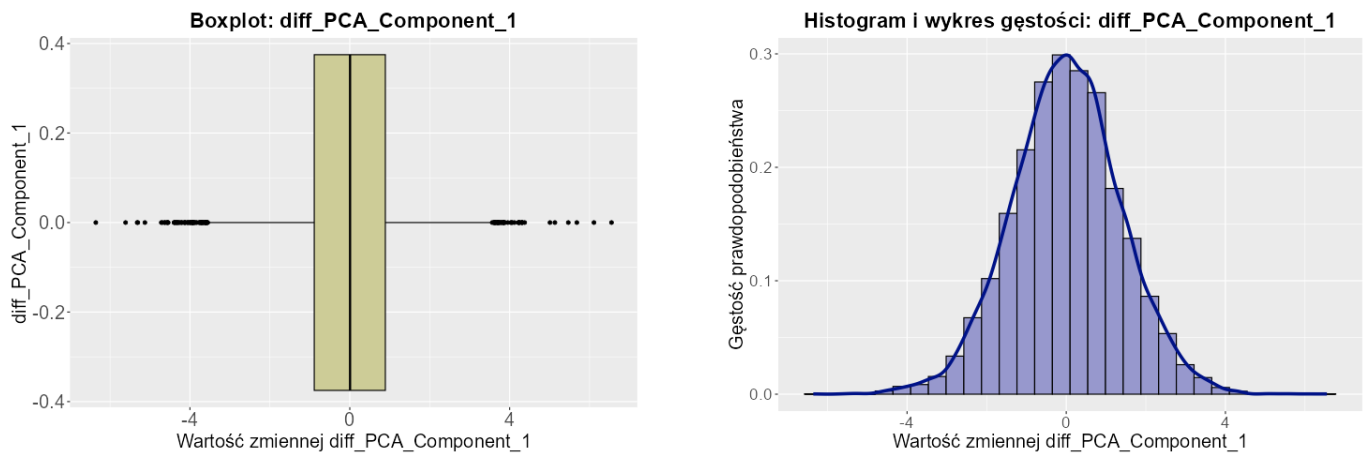


Fig. 15. Boxplot oraz histogram z nałożoną funkcją gęstości dla zmiennej diff_PCA_Component_1.

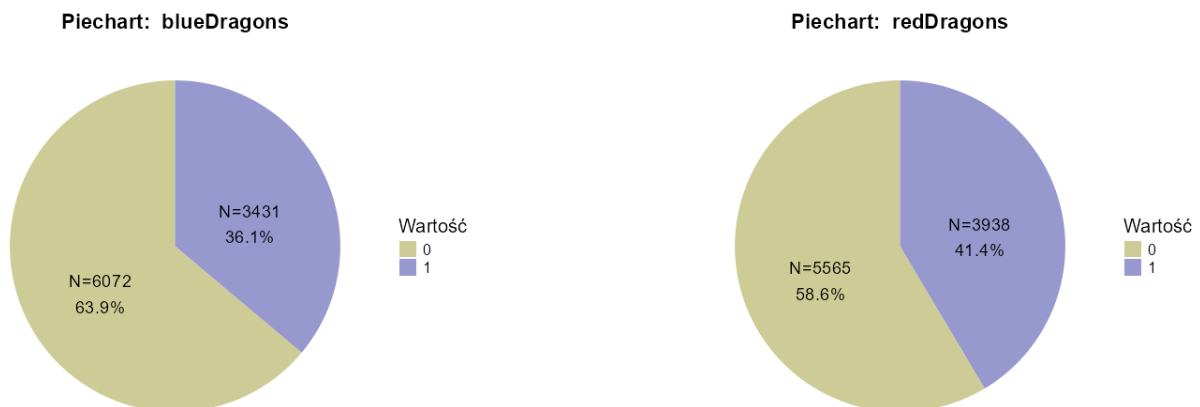


Fig. 16. Piecharty dla zmiennych blueDragons oraz redDragons.

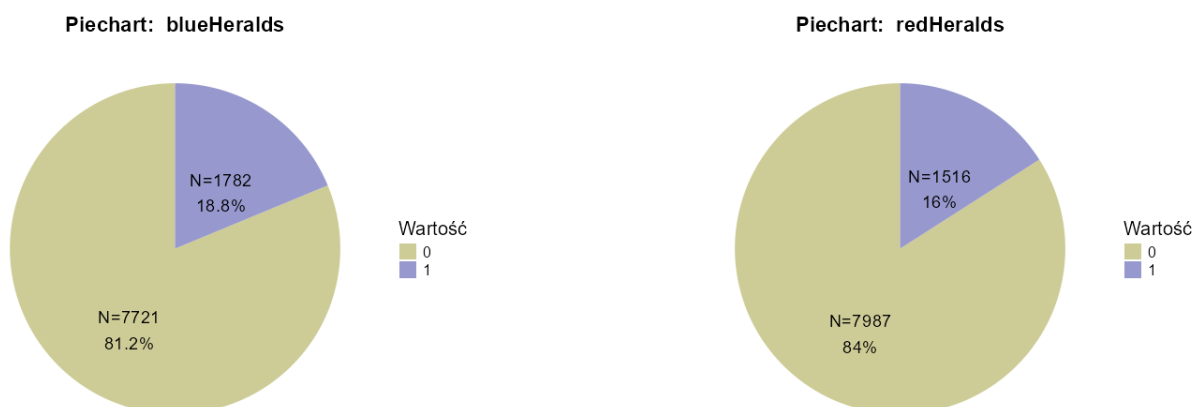


Fig. 17. Piecharty dla zmiennych blueHeralds oraz redHeralds.

Piechart: redFirstBlood

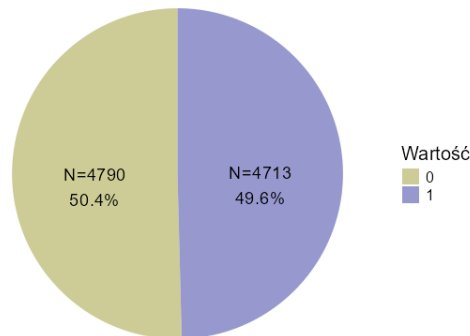


Fig. 18. Piechart dla zmiennej redFirstBlood.

5. Opisy metod

5.1. Sieci neuronowe

5.1.1. Opis

Sieci neuronowe [?] należą do grupy nieliniowych modeli klasyfikacyjnych inspirowanych strukturą biologicznych sieci neuronowych. Ich zadaniem jest aproksymacja funkcji odwzorowującej przestrzeń wejściową cech na zmienną decyzyjną. W analizie wykorzystano wielowarstwową perceptronową sieć neuronową (MLP) zaimplementowaną w bibliotekach `keras3` oraz `tensorflow` w języku R.

Rozważana sieć neuronowa składa się z warstw gęstych (ang. *dense layers*), w których każdy neuron jest połączony ze wszystkimi neuronami warstwy poprzedniej. Dla problemu klasyfikacji binarnej, model realizuje funkcję:

$$\hat{y} = \sigma(f(\mathbf{x}, \theta)) \quad (1)$$

gdzie:

- $\mathbf{x}$ – wektor cech wejściowych,
- θ – wektor parametrów modelu,
- $f(\cdot)$ – nieliniowa transformacja realizowana przez kolejne warstwy sieci,
- $\sigma(\cdot)$ – funkcja sigmoidalna:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

W analizie zastosowano architekturę składającą się z warstw ukrytych o liczbie neuronów 128 oraz 64. W warstwach ukrytych użyto funkcji aktywacji typu ELU oraz Softplus, które wprowadzają nieliniowość do modelu i pozwalają na modelowanie złożonych zależności w danych.

W celu ograniczenia przeuczenia zastosowano mechanizm *dropout*, który polega na losowym wyłączaniu części neuronów podczas uczenia z prawdopodobieństwem $p = 0.5$.

Model uczony był poprzez minimalizację funkcji straty typu binary cross-entropy:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (3)$$

gdzie:

- N – liczba obserwacji,
- y_i – rzeczywista klasa,
- $\hat{y}_i$ – predykcja modelu.

Optymalizacja parametrów modelu została przeprowadzona przy użyciu algorytmu RMSprop, który adaptacyjnie dostosowuje współczynnik uczenia na podstawie historii gradientów.

5.1.2. Procedura uczenia i ewaluacji

Dane zostały losowo podzielone na zbiór treningowy (60%) oraz testowy (40%). Proces uczenia był powtarzany 100 razy, aby ograniczyć wpływ losowej inicjalizacji wag oraz podziału danych na wynik końcowy.

Dla każdej iteracji model trenowano przez 50 epok przy stałej wielkości batcha wynoszącej 100 obserwacji. Jako miarę jakości klasyfikacji zastosowano dokładność (Binary Accuracy), a końcowy wynik stanowiła średnia ze 100 uruchomień modelu.

Skuteczność modelu oceniano na zbiorze testowym, co pozwoliło na uzyskanie stabilnej estymacji jakości klasyfikacji dla analizowanego problemu przewidywania wyniku meczu gry *League of Legends*.

5.2. Drzewo klasyfikacyjne

5.2.1. Opis

Drzewo decyzyjne [?] jest metodą klasyfikacji wykorzystującą strukturę hierarchiczną składającą się z węzłów decyzyjnych oraz liści reprezentujących klasy. Celem algorytmu jest podział przestrzeni cech na obszary możliwie jednorodne pod względem zmiennej decyzyjnej.

Proces budowy drzewa rozpoczyna się od umieszczenia wszystkich obserwacji w węźle korzeniowym. Następnie wykonywane są kolejne podziały zbioru danych na podstawie wartości wybranych cech. W każdym kroku wybierany jest taki podział, który prowadzi do największego zwiększenia jednorodności klas w otrzymanych podzbiorach.

Algorytm budowy drzewa można przedstawić w postaci następujących kroków:

1. Umieszczenie wszystkich obserwacji w węźle korzeniowym.
2. Wyznaczenie najlepszego podziału dla każdej dostępnej cechy.
3. Wybór podziału maksymalizującego jakość rozdzielania klas.
4. Utworzenie nowych węzłów potomnych.
5. Powtarzanie kroków 2–4 aż do spełnienia kryterium zatrzymania.

W analizie wykorzystano drzewo klasyfikacyjne służące do przewidywania wartości zmiennej `blueWins`, określającej końcowy wynik meczu z perspektywy drużyny niebieskiej.

5.2.2. Kryterium podziału

Podczas budowy drzewa zastosowano kryterium dewiancji, wykorzystywane przez bibliotekę `tree`. Dla pojedynczego węzła dewiancja definiowana jest jako:

$$D = -2 \sum_{i=1}^K n_i \ln(p_i) \quad (4)$$

gdzie:

- K – liczba klas,
- n_i – liczba obserwacji należących do klasy i ,
- p_i – prawdopodobieństwo wystąpienia klasy i w analizowanym węźle.

Optymalny podział jest wybierany w taki sposób, aby maksymalnie zmniejszyć całkowitą dewiancję drzewa.

5.2.3. Przycinanie drzewa

Drzewa decyzyjne są podatne na przeuczenie (*overfitting*), szczególnie gdy ich struktura staje się zbyt rozbudowana. W celu ograniczenia tego zjawiska zastosowano proces przycinania drzewa (*pruning*).

Przycinanie polega na usuwaniu części gałęzi, które nie wnoszą istotnej informacji predykcyjnej. W pracy optymalny rozmiar drzewa został wyznaczony z wykorzystaniem walidacji krzyżowej. Spośród analizowanych struktur wybierano model minimalizujący błąd klasyfikacji na danych walidacyjnych.

5.3. Sieci bayesowskie

5.3.1. Opis

Sieci bayesowskie [?] są probabilistycznymi modelami graficznymi reprezentującymi zależności warunkowe pomiędzy zmiennymi losowymi. Struktura sieci ma postać acyklicznego grafu skierowanego (DAG), w którym węzły odpowiadają zmiennym, natomiast krawędzie opisują zależności warunkowe pomiędzy nimi.

Celem modelu jest reprezentacja wspólnego rozkładu prawdopodobieństwa w postaci iloczynu rozkładów warunkowych:

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^n P(X_i \mid Pa(X_i)) \quad (5)$$

gdzie:

- X_i – zmienna losowa,
- $Pa(X_i)$ – zbiór rodziców węzła X_i w grafie skierowanym.

W analizie wykorzystano sieć bayesowską do klasyfikacji zmiennej *blueWins*, określającej wynik meczu z perspektywy drużyny niebieskiej.

5.3.2. Uczenie struktury i parametrów

W pierwszym etapie przeprowadzono uczenie struktury sieci bayesowskiej na podstawie danych treningowych. W tym celu zastosowano algorytm wspinaczki (ang. *hill-climbing*, HC), który iteracyjnie modyfikuje strukturę grafu w celu maksymalizacji funkcji dopasowania modelu do danych.

Następnie estymowano parametry modelu, czyli rozkłady warunkowe zmiennych, na podstawie danych treningowych. Formalnie oznacza to estymację wartości:

$$P(X_i \mid Pa(X_i)) \quad (6)$$



dla każdej zmiennej w modelu.

5.3.3. Predykcja modelu

Predykcja zmiennej decyzyjnej `blueWins` została wykonana z wykorzystaniem ważonego estymatora bayesowskiego (ang. *likelihood weighting*). Dla każdej obserwacji testowej wyznaczano najbardziej prawdopodobną klasę:

$$\hat{y} = \arg \max_{c \in \{0,1\}} P(\text{blueWins} = c \mid \mathbf{x}) \quad (7)$$

gdzie $\mathbf{x}$ oznacza wektor zmiennych wejściowych.

5.4. Opis metody hybrydowej

5.5. Metody oceniania

Ocena jakości modeli klasyfikacyjnych stanowi istotny element analizy, ponieważ pozwala na porównanie skuteczności zastosowanych metod oraz określenie ich przydatności w kontekście rozważanego problemu predykcji wyniku meczu. W niniejszej pracy wykorzystano miary oparte na macierzy pomyłek oraz krzywą ROC wraz z polem pod krzywą AUC.

5.5.1. Macierz pomyłek i miary klasyfikacji

Podstawą oceny modeli klasyfikacyjnych jest macierz pomyłek, która zestawia rzeczywiste klasy obserwacji z klasami przewidzianymi przez model. Dla problemu binarnej klasyfikacji wyróżnia się cztery podstawowe wartości:

- *TP* (True Positive) – liczba poprawnie przewidzianych przypadków pozytywnej klasy,
- *TN* (True Negative) – liczba poprawnie przewidzianych przypadków negatywnej klasy,
- *FP* (False Positive) – liczba błędnie przewidzianych przypadków pozytywnej klasy,
- *FN* (False Negative) – liczba błędnie przewidzianych przypadków negatywnej klasy.

Na podstawie macierzy pomyłek definiuje się następujące miary jakości klasyfikacji:

Dokładność (Accuracy) określająca odsetek poprawnych klasyfikacji:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (8)$$

Precyzja (Precision) określająca, jak wiele przewidzianych przypadków pozytywnych jest rzeczywiście poprawnych:

$$Precision = \frac{TP}{TP + FP} \quad (9)$$

Czułość (Sensitivity, Recall) określająca zdolność modelu do wykrywania pozytywnych przypadków:

$$Sensitivity = \frac{TP}{TP + FN} \quad (10)$$

Specyficzność (Specificity) określająca zdolność modelu do poprawnego wykrywania przypadków negatywnych:

$$Specificity = \frac{TN}{TN + FP} \quad (11)$$



Negatywna wartość predykcyjna (NPV) określająca poprawność przewidywań klasy negatywnej:

$$NPV = \frac{TN}{TN + FN} \quad (12)$$

Miary te pozwalają na kompleksową ocenę jakości modeli, uwzględniając zarówno poprawne klasyfikacje, jak i błędy różnych typów.

5.5.2. Krzywa ROC oraz pole pod krzywą AUC

Krzywa ROC (Receiver Operating Characteristic) przedstawia zależność pomiędzy czułością modelu a odsetkiem fałszywych alarmów (False Positive Rate):

$$FPR = \frac{FP}{FP + TN} \quad (13)$$

Krzywa ROC jest wykresem funkcji:

$$ROC = (FPR, Sensitivity) \quad (14)$$

Analiza krzywej ROC pozwala ocenić kompromis pomiędzy czułością a specyficznością modelu dla różnych progów decyzyjnych.

Pole pod krzywą ROC (AUC – Area Under Curve) stanowi zagregowaną miarę jakości klasyfikatora:

$$AUC = \int_0^1 TPR(FPR) d(FPR) \quad (15)$$

gdzie TPR oznacza czułość modelu.

Wartość AUC przyjmuje wartości z przedziału $[0, 1]$, gdzie wartość 1 oznacza idealny klasyfikator, natomiast wartość 0,5 odpowiada klasyfikatorowi losowemu. W niniejszej pracy miara AUC została wykorzystana do porównania skuteczności analizowanych modeli klasyfikacyjnych.



6. Rezultaty

6.1. Rezultat metody 1

6.2. Rezultat metody 2

6.3. Rezultat metody 3

6.4. Rezultat metody hybrydowej

6.5. Porównanie skuteczności metod

6.6. Użycie najlepszego modelu na syntetycznych danych

7. Podsumowanie

8. Załączniki

Literatura

- [1] International Energy Agency, “World energy outlook 2023.” <https://www.iea.org/reports/world-energy-outlook-2023>, 2023. Last accessed: March 2026.